

Subject: SAST Summer Training 2026, Agent Harness
Date: from 2026/10/14 to 2026/10/15

Contents

Multi-Agent and Compaction	1
Multi-Agent	1
多层 Compact	2
Summary	4

Multi-Agent and Compaction

Multi-Agent

Agent 与 Task

Multi-Agent 中要区分两个概念：**Agent 是执行者，Task 是协调记录**. 父 Agent 负责拆分目标，子 Agent 在受控上下文中执行，Task 工具记录负责人、状态和依赖.

父 Query

- > Agent tool_use
- > AgentTool 解析 prompt / subagent_type / model / 运行模式
- > 构造子 Agent 的 Query 和工具上下文
- > 子 Agent 执行搜索、读取、编辑或测试
- > 返回摘要、状态或错误
- > 父 Query 收到 tool_result, 继续决策

子 Agent 的上下文

子 Agent 通常拥有独立的消息历史、agentId、工具池、权限和取消控制器，但可以继承父任务所需的上下文、MCP 连接或工作目录信息. 源码重点位于 src/tools/AgentTool/、src/tasks/LocalAgentTask/ 和 src/tasks/RemoteAgentTask/.

常见执行模式

模式	说明	父 Agent 得到的结果
前台 / 同步	当前进程等待子 Query 完成，适合短分析任务	completed 或错误结果
本地后台	注册 LocalAgentTask, 子 Agent 独立运行，适合长任务和并行探索	async_launched, 完成后通知
Fork / Worktree	Fork 派生上下文; Worktree 创建隔离 Git 工作副本，二者可以组合	子任务结果、任务状态或变更路径
Remote	创建远程会话并由 RemoteAgentTask 轮询，适合远程或长期任务	remote_launched、session URL

Task 工具管理的是工作项

TaskCreate、TaskUpdate、TaskList 和 TaskGet 管理的是工作项，不会自动启动 Agent:

TaskCreate	创建 pending 任务，暂时没有 owner
TaskUpdate	分配 owner, 更新状态和 blocks / blockedBy 依赖
TaskList / TaskGet	查看可执行任务或单项详细状态
Agent / teammate	领取任务并真正运行子 Query
TaskUpdate(completed)	回写完成状态，解除后续任务的阻塞

状态流转与依赖

通常使用 `pending -> in_progress -> completed` 的状态流转；`deleted` 表示永久移除任务。`addBlockedBy: ["1"]` 表示当前任务必须等待任务 1，系统也会把反向关系写入任务 1 的 `blocks`。TeamCreate 会建立团队上下文和对应的 TaskList，但仍需通过 TaskCreate 创建工作项、TaskUpdate 分配负责人。

Multi-Agent 的核心

因此，Multi-Agent 的核心不是简单地“多开几个模型”，而是同时管理父子委托、上下文与权限隔离、任务依赖、取消传播和结果汇总。只读分析可以并行；共享文件编辑应串行，或使用独立 Worktree。

多层 Compact

Compact 的分层

Claude Code 并不只有一种 Compact。源码把上下文收缩做成多个层次：先处理最占空间、最容易安全丢弃的工具结果，再尝试局部历史压缩；只有仍然接近窗口上限时，才重建整段会话。讲解时还要把“怎样压缩”和“何时触发”分开：`/compact`、`auto compact` 和 `reactive compact` 是触发路径，不完全是三种彼此独立的摘要算法。

按处理粒度分类

层次	机制	主要行为	是否调用模型生成摘要
工具结果	Tool-result budget / content replacement	对过大的工具结果实施预算限制，以占位内容替换或外置原始结果	否
工具结果	Microcompact	清除较早、可压缩的 <code>tool_result</code> ，保留最近结果和消息结构	否
局部历史	History snip	删除被选中的旧消息片段，并记录 <code>snip boundary</code> ；可与 <code>microcompact</code> 同时发生	否
局部历史	Context collapse	将较早的会话片段逐段归档为摘要，并在请求时投影出压缩视图	是
整段会话	Session Memory compact	优先复用后台持续提取的 <code>Session Memory</code> ，再拼接尚未总结的近期消息	通常不重新总结
整段会话	Traditional compact	调用模型总结旧历史，然后用摘要、保留消息和重新注入的上下文重建会话	是

microcompact 的两个分支

- **Time-based microcompact.** 若主线程距离上次模型回复已经很久，服务端 Prompt Cache 通常已经变冷，因此直接清空较早的可压缩工具结果，只保留最近若干项。
- **Cached microcompact.** 在模型和服务端支持 cache editing 时，按 tool_use_id 提交 cache_edits 删除缓存中的旧工具结果；本地 transcript 可以保持不变，从而避免为了缩短上下文而重写整个缓存前缀。

collapse 与 Session Memory

Context collapse 与 traditional compact 也不能视为同一个步骤。Context collapse 以较小片段增量归档历史，摘要保存在独立的 collapse store 中，请求时再投影为可见消息；traditional compact 则用一次完整摘要替换主要历史。启用 Context collapse 时，主动 auto compact 可能被抑制，以免一个全局摘要覆盖刚建立的细粒度折叠结果。

Session Memory compact 是完整 Compact 的另一条实现路径。执行手动或自动 Compact 时，系统会先尝试读取已经持续维护的 Session Memory，并保留总结边界之后的近期消息；若 Memory 不存在、为空、边界无效，或压缩后仍超过自动阈值，再退回 traditional compact。

上述分支受版本、模型能力、用户类型和 feature flag 影响，并非每个发行构建都会同时启用。

按触发方式分类

触发方式	触发时机	可能采用的实现
手动 Compact	用户执行 /compact [自定义指令]	无自定义指令时先尝试 Session Memory compact，否则或失败后使用 traditional / reactive 路径
Auto compact	请求模型前估算 Token，达到预留输出空间后的阈值	先尝试 Session Memory compact，再回退到 compactConversation()；连续失败会触发熔断
Reactive compact	模型 API 已返回 prompt too long、HTTP 413 或媒体过大错误	prompt-too-long 时先尝试排空可提交的 context collapse；媒体错误跳过该步骤，直接进入剥离、压缩与重试路径

请求前的处理顺序

因此，请求前的上下文处理更接近下面的顺序：

完整历史

- > tool-result budget
- > history snip (若启用)
- > microcompact
- > context collapse (若启用)
- > auto compact 阈值检查

-> 调用模型
|- 若上下文或媒体错误: `collapse drain / reactive compact` -> 重试

完整摘要与重建

`autoCompactIfNeeded()` 负责主动压缩的阈值判断和连续失败熔断. 真正执行完整摘要时, `compactConversation()` 会运行 `PreCompact hooks` 并生成 `CompactionResult`; `buildPostCompactMessages()` 按以下顺序构造下一轮历史:

Compact boundary
-> `summaryMessages`
-> `messagesToKeep`
-> `attachments`
-> `hookResults`

Compact 的核心

Compact 完成后不会结束 Agent Loop, 而是把压缩后的消息写回循环状态并继续请求模型. 它的核心不是单纯“让历史变短”, 而是重建一个仍可继续执行的状态; 若摘要遗漏用户目标、已完成修改、失败原因或后续计划, Agent 就可能重复工作或做出错误决策.

Summary

Harness 的闭环

一个完整的 Agent Harness 通常围绕以下闭环工作:

收集输入与环境
-> 构建上下文并调用模型
-> 解析文本或 `tool_use`
-> 校验权限并执行工具
-> 将 `tool_result` 写回消息历史
-> 继续下一轮, 直到模型给出最终回答

模型与 Harness 的分工

其中, 模型决定“下一步做什么”, 而 Harness 决定“这一步能否执行、如何执行、如何记录, 以及如何把结果交还给模型”.